UTN

BASE DE DATOS
FOOD STORE

Tecnicatura Universitaria en Programación

Integrantes

- Mendez, Maximiliano
- Sanchez, Gonzalo
- Fachinelli, Leonel
- Ávila Román
- Danna, Gabriel
- Ortiz, Mailen

24/08/2026

Índice

Marco Teórico **2**
Modelo Relacional **2**
Normalización **2**
1) Abstracción del Dominio y Nivel Conceptual 2
2) Derivación del Esquema Lógico y Relaciones 3
Integridad **3**
Transacciones **4**
Diagramas **5**
DER Conceptual **5**
Diccionario de entidades y atributos 6
Esquema Relacional // Físico **9**
Decisiones de Diseño **10**
Normalización 10
Manejo de total 10
Soft Delete 10
Índices 11
Capturas **12**
Ejecución de consultas clave 12
EXPLAIN ANALYZE 14
Escenarios de concurrencia 14
Dificultades y soluciones **16**
Bibliografía/Webgrafía 16
Links al repo 16

Marco Teórico

Modelo Relacional

El modelo relacional es una forma de organizar y gestionar datos en una base de datos mediante tablas con filas y columnas. Fue creado por Edgar F. Codd en 1970 y constituye la base de la mayoría de los sistemas modernos de gestión de bases de datos relacionales.

El modelo relacional nos permite conectar datos entre diferentes tablas mediante relaciones. Los datos que se almacenan en estas se pueden conectar entre sí mediante claves, lo que facilita la organización, consulta y gestión de grandes cantidades de datos estructurados. El acceso y la gestión de los datos se realizan mediante SQL

Entidades Fuertes y Atributos presentados en el diagrama MER:

- **Entidades:** Representan las clases de objetos del dominio con existencia propia: USUARIO, CATEGORIA, PRODUCTO, PEDIDO y DETALLE_PEDIDO.
- **Atributos Comunes:** Todas las entidades incluyen id (clave primaria subrogada de tipo bigint), eliminado (boolean para soporte de borrado lógico / *soft delete*) y created_at (timestamptz para auditoría temporal). Al no contar el modelo relacional con herencia de implementación pura, la repetición deliberada de estas tres columnas en cada tabla es la práctica estándar por simplicidad y rendimiento.
- **Vínculos Lógicos** → Las entidades no se asocian mediante punteros en memoria, sino a través del emparejamiento de valores mediante claves.

Normalización

El diseño de la base de datos para el sistema de gestión de pedidos de comida *Food Store* se encuadra dentro de los principios del modelo relacional, abordando de forma sucesiva los niveles conceptual y lógico para mantener una buena organización y eficiencia.

1) Abstracción del Dominio y Nivel Conceptual

El modelo conceptual junta las entidades principales del negocio. El dominio se estructura a partir de cinco entidades fuertes principales: USUARIO, CATEGORIA, PRODUCTO, PEDIDO y DETALLE_PEDIDO.

Estructura de Atributos Comunes y Ausencia de Herencia Relacional

A diferencia del paradigma orientado a objetos, donde las propiedades compartidas se resuelven mediante herencia de clases, el modelo relacional estándar carece de un mecanismo eficiente.

Identificador Surrogado (id): Clave primaria autogenerada (BIGINT) desprovista de significado de negocio, cuya función es simplificar el establecimiento de claves foráneas y la indexación.

- **Borrado Lógico (eliminado):** Indicador booleano que sustituye las eliminaciones físicas por un estado de inhabilitación, garantizando la trazabilidad y la preservación del historial.
- **Auditoría Temporal (created_at):** Estampa de tiempo con zona horaria que registra el momento preciso de la salida en la base de datos.

2) Derivación del Esquema Lógico y Relaciones

El paso del nivel conceptual al esquema relacional exige traducir los vínculos del diagrama Entidad-Relación mediante la inclusión de claves foráneas (FK) según las direcciones identificadas:

1. **Relación USUARIO – PEDIDO ($1:N$):** Un usuario puede efectuar múltiples compras, pero cada pedido pertenece unívocamente a un usuario. Se implementa incorporando usuario_id en la tabla PEDIDO.
2. **Relación CATEGORIA – PRODUCTO ($1:N$):** Una categoría agrupa múltiples ítems del catálogo. Se muestra con el atributo categoria_id en la tabla PRODUCTO.
3. **Relación PRODUCTO – DETALLE_PEDIDO ($1:N$):** Cada producto puede figurar en diversas líneas de venta. Se muestra con la clave foránea producto_id en la tabla DETALLE_PEDIDO.
4. **Composición PEDIDO – DETALLE_PEDIDO ($1:N$):** Existe una **dependencia de existencia** entre el pedido y sus líneas; el detalle carece de sentido fuera de su cabecera. Esta relación requiere el atributo pedido_id en DETALLE_PEDIDO. Además, para evitar redundancias dentro de una misma transacción, se aplica una restricción de unicidad compuesta.

Integridad

En el modelo de datos relacional, la integridad de los datos garantiza que la información que se almacena en una base de datos sea confiable, precisa y consistente. Para mantener esa integridad las bases de datos relacionales utilizan restricciones, que son reglas aplicadas a tablas y columnas, las cuales impiden que se almacenen datos no válidos o inconsistentes.

➢ **Integridad de Entidad (Primary Key - PK):** Identifica a cada tupla dentro de una tabla y prohíbe valores nulos. Se utilizan claves subrogadas autoincrementales (BIGINT) en todas las tablas (id).

➢ **Integridad Referencial (Foreign Key - FK):** Asegura que el valor de un atributo que apunta a otra tabla coincida con un valor existente de la clave primaria referenciada.

Se definen políticas explícitas como ON DELETE RESTRICT para evitar la eliminación en cascada de pedidos históricos y forzar el uso de borrado lógico.

➢ **Integridad de Dominio y Restricciones Semánticas:**
- Restricciones **NOT NULL** para obligatoriedad de datos, el campo no puede quedar vacío.

- Restricciones **UNIQUE** para evitar duplicados en campos representativos (como usuario.mail o categoria.nombre) y la unicidad compuesta UNIQUE(pedido_id, producto_id) en líneas de pedido.
- Restricciones **CHECK** para validar dominios de valor admisibles (precio >= 0, stock >= 0, cantidad > 0, total >= 0).
- Tipos de datos enumerados (**ENUM**) para acotar los dominios de rol, estado_pedido y forma_pago.

Transacciones

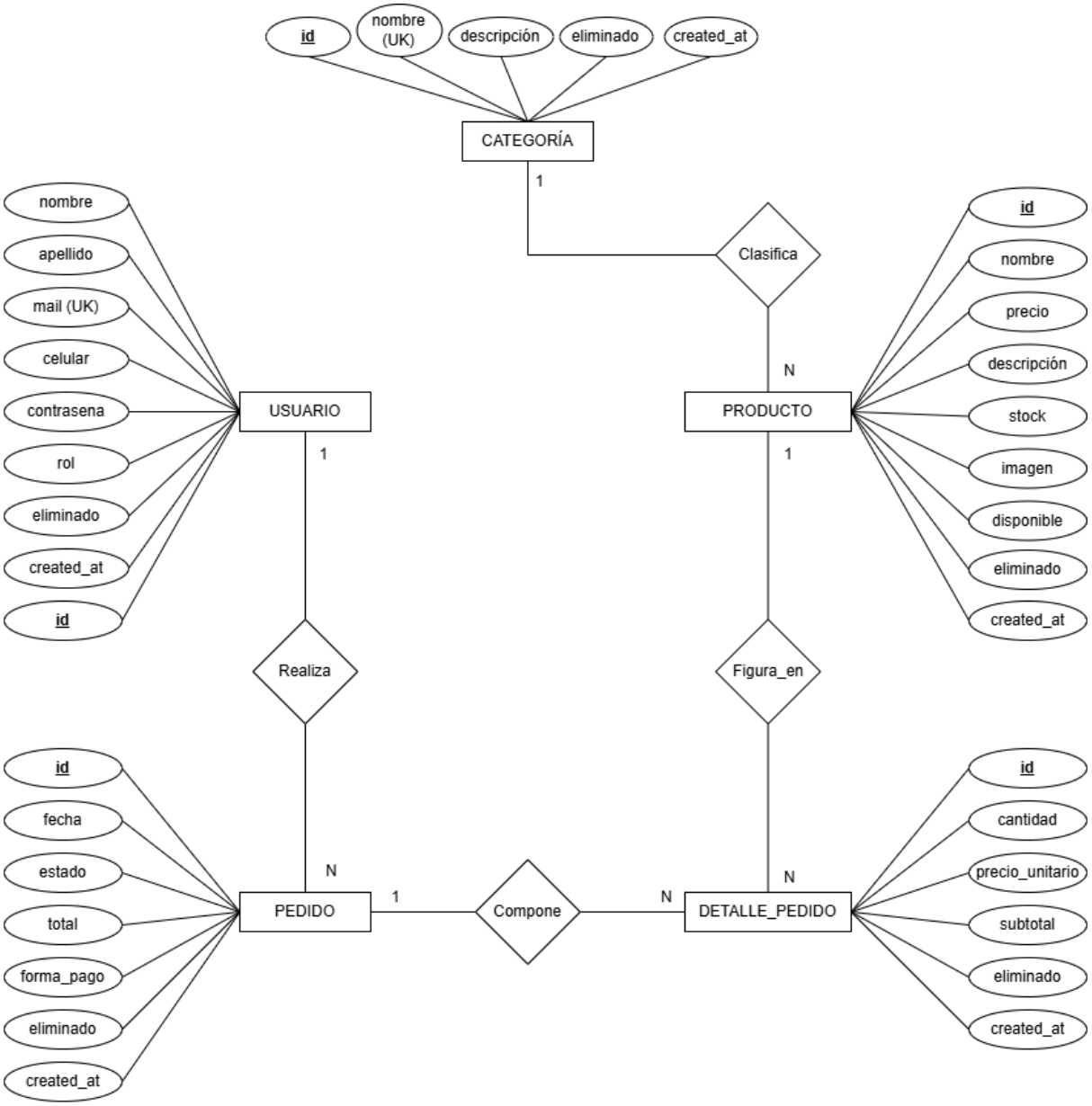
Una Transacción es una secuencia de una o más operaciones sobre la base de datos (INSERT, UPDATE, DELETE) que se ejecuta como una unidad indivisible: se confirman todos sus cambios o ninguno. Esta garantía se resume en las propiedades ACID

- **Atomicidad**: La transacción es todo o nada; si una operación falla, se revierten con ROLLBACK los cambios ya realizados.
- **Consistencia**: La base de datos pasa de un estado válido a otro, respetando las restricciones definidas como las claves foráneas, checks, etc.
- **Aislamiento**: Las transacciones concurrentes no interfieren entre sí; el nivel de aislamiento configurado determina qué cambios de una transacciones son visibles para otra mientras ambas están en curso.
- **Durabilidad**: Una vez confirmada con COMMIT, la transacción persiste aunque ocurra una falla posterior del sistema.

En el proyecto, estas propiedades se implementan mediante bloques BEGIN, COMMIT, ROLLBACK y, principalmente, a través del procedimiento **sp_crear_pedido**, que agrupa la creación del pedido y el descuento de stock en una única transacción: si algún ítem es invalido o no hay stock suficiente, todo la operación se revierte sin dejar registro huérfanos. Además, se utiliza **SELECT … FOR UPDATE** para bloquear la fila del producto durante la operación y evitar la sobreventa entre usuarios concurrentes, y se comparan los niveles de aislamiento READ COMMITTED y SERIALIZABLE para mostrar cómo varía la visibilidad de los datos entre transacciones simultáneas.

Diagramas

DER Conceptual



¿Por qué la relación entre producto y pedido no puede resolverse como una relación binaria simple 1:N? ¿Qué información se perdería si lo intentaras?

La relación entre producto y pedido no pueden resolverse como una relación simple de 1:N ya que por ejemplo: si pusiéramos **producto_id** en PEDIDO estaremos ligando Un pedido a un solo producto, de esta manera, limitando a que un solo pedido tenga un solo producto.

Y si pusieramos **pedido_id** en PRODUCTO estaremos ligando un producto a un solo pedido, de esta manera, restringiendo un único pedido a un solo producto.

Además, perderíamos toda la información sobre los atributos almacenados en la tabla intermedia DETALLE_PEDIDO como la cantidad, subtotal, etc.

¿Qué entidad tiene participación parcial en la relación con categoría, y cuál tiene participación total? ¿Qué pasaría si invirtieras esa lectura?

En la relación entre PRODUCTO y CATEGORÍA la entidad que tiene participación parcial es categoría, ya que una categoría puede existir de forma independiente, sin tener todavía asignado a ningún producto. En cambio, un producto al momento de su existencia, debe poder tener asignada una categoría.

Lo que pasaría si invirtiéramos esa lectura (la categoría tiene participación total) es que nos obligaría a crear un producto en el mismo momento en que creamos una categoría. Además podríamos tener productos sueltos sin categorías.

¿Alguno de tus atributos podría descomponerse en partes más simples (por ejemplo, un nombre completo en nombre y apellido)? Justificá tu decisión de dejarlo compuesto o de separarlo.

En el momento de diseñar, decidimos descomponer el nombre de los usuarios en dos atributos distintos: nombre y apellido. Ya que, dejarlo compuesto limitaría al sistema. Al tenerlos separados, la base de datos es mucho mas eficiente y flexible, permitiendonos realizar consultas, filtrados, etc.

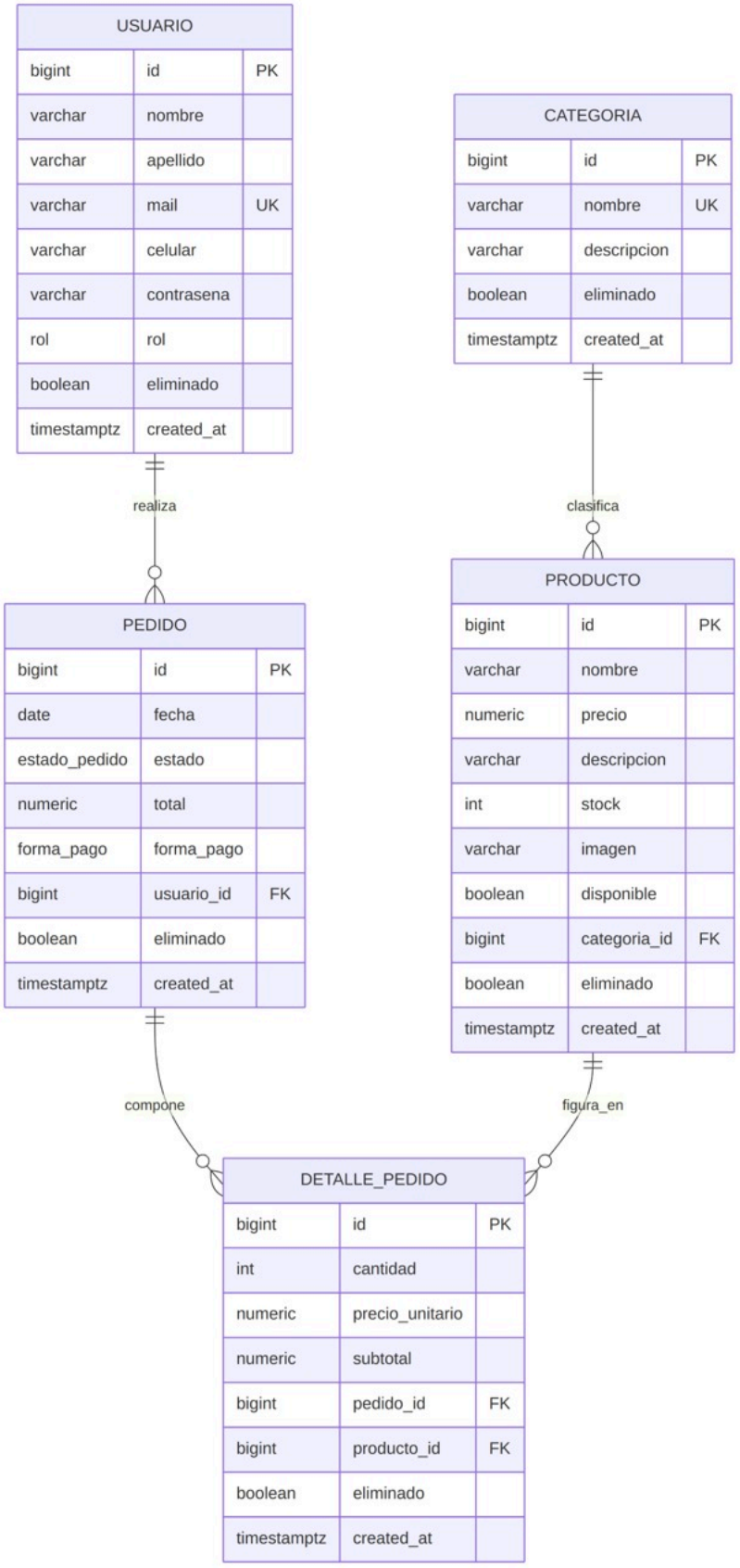
Diccionario de entidades y atributos

Entidad	Atributo	Tipo de Dato	Tipo de Clave Restricción	Descripción
CATEGORÍA	Id	Numérico (Entero)	PK (Clave primaria)	Identificador único de la categoría
	nombre	Texto	UK (Clave candidata/Única), Obligatorio	Nombre representativo de la categoría
	descripción	Texto		Detalle o información complementaria
	eliminado	Booleano		Borrado lógico
	created_at	Fecha y hora		Fecha y hora de registro
USUARIO	id	Numérico (Entero)	PK (Clave primaria)	Identificador único de usuario
	nombre	Texto		Nombre(s) del usuario

	apellido	Texto		Apellido(s) del usuario
	mail	Texto	UK (Clave candidata/Única), Obligatorio	Correo electrónico de acceso y contacto
	celular	Texto		Teléfono de contacto
	contrasena	Texto		Contraseña de acceso
	rol	Texto		Rol o permisos dentro del sistema (ej. Cliente, Admin)
	eliminado	Booleano		Borrado Lógico
	created_at	Fecha y hora		Fecha y hora de alta en el sistema
PRODUCTO	id	Numérico (Entero)	PK (Clave primaria)	Identificador único del producto
	nombre	Texto		Nombre comercial del producto
	precio	Numérico (Decimal)		Precio actual de venta unitario
	descripción	Texto		Descripción detallada de las características
	stock	Numérico (Entero)		Unidades disponibles para la venta
	imagen	Texto		Enlace o referencia a la imagen del producto
	disponible	Booleano		Indica si el producto está visible para la venta
	eliminado	Booleano		Borrado lógico
	created_at	Fecha y hora		Fecha y hora de registro del producto
PEDIDO	id	Numérico (Entero)	PK (Clave primaria)	Identificador único de la orden de compra
	fecha	Fecha y hora		Momento exacto en que se realiza la compra
	estado	Texto		Estado de la orden (ej. Pendiente, En preparación, Entregado)
	total	Numérico (Decimal)		Monto final total de la compra

	forma_pago	Texto		Método de pago utilizado (ej. Tarjeta, Efectivo)
	eliminado	Booleano		Borrado lógico
	created_at	Fecha y hora		Fecha y hora de creación de la orden
DETALLE_PEDIDO	id	Numérico (Entero)	PK (Clave primaria)	Identificador único de la línea/ítem de compra
	cantidad	Numérico (Entero)		Unidades compradas de dicho producto
	precio_unitario	Numérico (Decimal)		Precio al momento exacto de la compra
	subtotal	Numérico (Decimal)		Monto total de la línea
	eliminado	Booleano		Borrado lógico
	created_at	Fecha y hora		Fecha y hora de registro de la línea

Esquema Relacional // Físico



Decisiones de Diseño

Normalización

El diseño satisface la Forma Normal de Boyce-Codd (BCNF), resolviendo tres problemas críticos del modelado:

- **Preservación de la Fidelidad Histórica:** Aunque el precio de un artículo se define en PRODUCTO, la entidad DETALLE_PEDIDO incluye de forma directael atributo precio_unitario. Esto incorpora una copia estatica del valor del precio unitario al momento de la venta, impidiendo que futuras modificaciones en el catálogo alteren los registros de facturaciones anteriores.
- **Desnormalización Controlada de Datos Derivados:** Los campos subtotal y total son datosque pueden calcularse. Bajo un criterio de normalización estricto no deberían almacenarse; sin embargo, decidimos por optar por una **desnormalización**, materializandose en columnas e implementando procedimientos almacenados y disparadores (*triggers*) para su cálculo automático. Esto prioriza el rendimiento de las consultas frecuentes de agregación sobre el costo de almacenamiento.
- **Satisfacción Trivial de Formas Normales:** Al definirse una clave surrogada simple (id) como clave primaria en cada tabla, se garantiza de forma trivial el cumplimiento de la **Segunda Forma Normal (2FN)**, al no existir dependencias parciales respecto de claves compuestas.

Manejo de total

Manejo de total y consistencia de datos, la integridad y el rendimiento en el cálculo de montos financieros, se adoptaron las siguientes estrategias:

- **Cálculo descentralizado:** La lógica del cálculo de totales se delega a disparadores (*triggers*) de servidor sobre la tabla **DETALLE_PEDIDO**. Ante cualquier inserción, modificación o eliminación de un ítem, el sistema recalcula automáticamente el **SUBTOTAL** de la linea y actualiza el **TOTAL** de la cabecera del **PEDIDO.**
- **Integridad Transaccional:** Se garantiza que el valor **TOTAL** almacenado sea siempre una representación exacta de la suma de sus partes. Al materializar este cálculo en la base de datos, se elimina la necesidad de realizar operaciones de agregación costosas (**SUM()**) en cada consulta de lectura, mejorando significativamente el rendimiento en reportes y vistas de usuario.

Soft Delete

Para continuar con el rendimiento y la estructura del proyecto se implementa una estrategia de borrado lógico en todas las entidades del sistema:

- **Estado de Inhabilitación:** En lugar de ejecutar sentencias **DELETE** físicas que removerían permanentemente los registros, el sistema utiliza un campo booleano **ELIMINADO** (con valor por defecto **FALSE**). Al "borrar" un registro, simplemente se actualiza este campo a **TRUE**.
- **Filtros por Defecto:** Se establece como regla de negocio que todas las consultas de lectura (*SELECT*) incluyan implícitamente la condición **WHERE eliminado = FALSE**. Esto garantiza que los registros "borrados" permanezcan invisibles para la operación cotidiana del usuario sin sacrificar la persistencia.
- **Trazabilidad y Auditoría:** Esta decisión permite mantener la integridad referencial de datos históricos (por ejemplo, un producto eliminado no debe desaparecer de un **DETALLE_PEDIDO** pasado). Además, ofrece una capa de seguridad operativa, permitiendo una restauración rápida de información sin necesidad de recurrir a copias de seguridad (*backups*) completas de la base de datos.

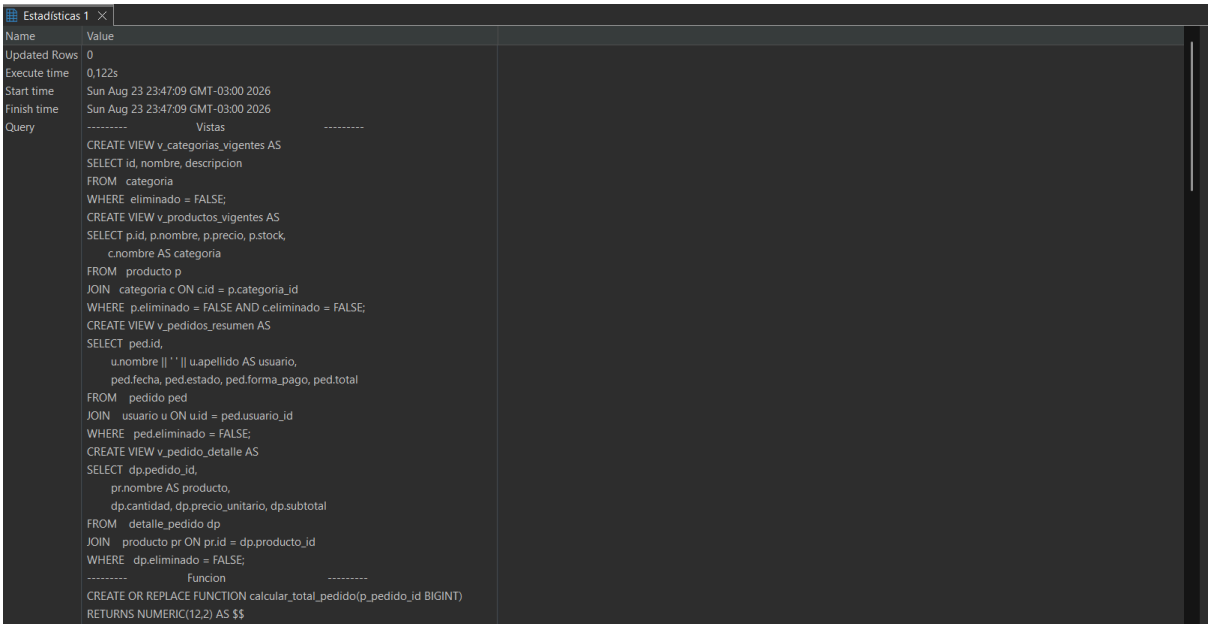
Índices

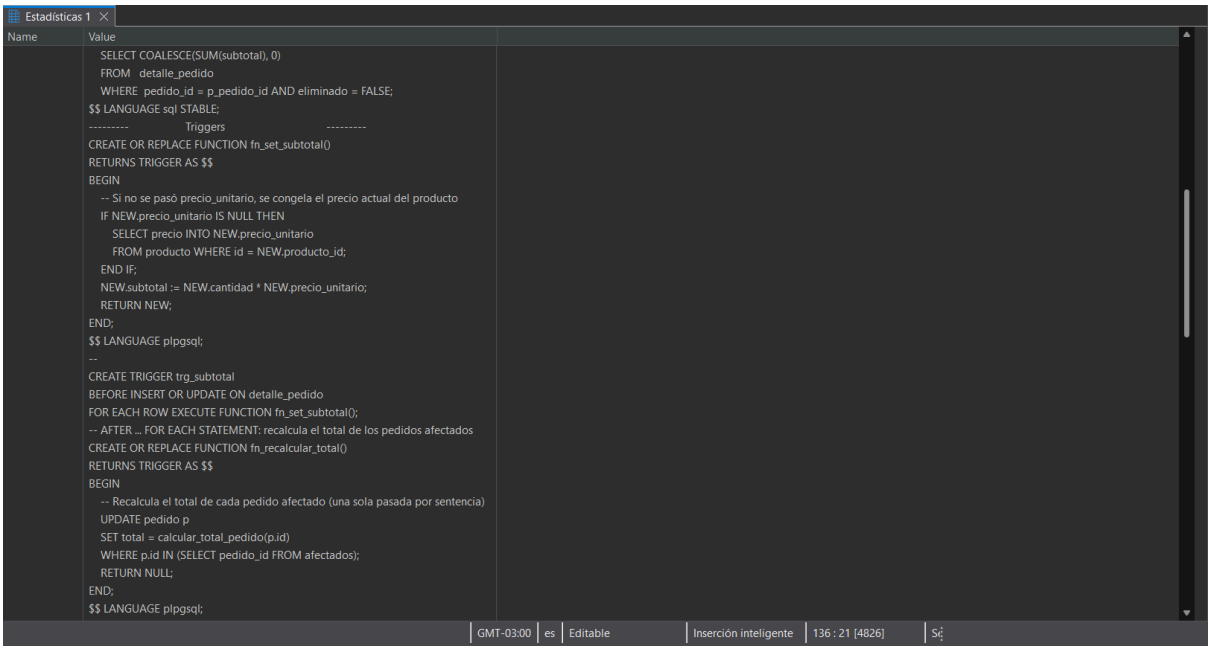
Optimización de Acceso y Rendimiento: Con el fin de agilizar las operaciones de búsqueda y reducir los tiempos de latencia en las consultas de mayor recurrencia, se ha implementado una arquitectura de índices selectivos:

- **Indexación sobre Claves Foráneas (FK):** Se han aplicado índices en todos los campos de clave foránea (por ejemplo: **usuario_id, categoria_id, producto_id**). Esto acelera drásticamente las operaciones de *JOIN* y permite una navegación eficiente entre entidades relacionadas.
- **Índices en Campos de Filtrado Frecuente:** Se han creado índices específicos en columnas utilizadas recurrentemente en cláusulas WHERE, como email (para login de usuarios) y eliminado (en tablas de alto volumen). Esto reduce el costo de escaneo secuencial (*table scan*) y favorece el uso de *index scans*.
- **Índices Compuestos:** Para consultas que combinan múltiples criterios, se implementaron índices compuestos. Un caso crítico es UNIQUE(**pedido_id, producto_id**) en la tabla **DETALLE_PEDIDO**, que no solo garantiza la integridad de los datos evitando duplicados, sino que también optimiza las búsquedas específicas de productos dentro de un pedido determinado.
- **Política de Minimización:** Se ha evitado el exceso de índices para no degradar el rendimiento en las operaciones de escritura (*INSERT*, *UPDATE*, *DELETE*), asegurando que el *overhead* de mantenimiento de los índices se mantenga dentro de márgenes aceptables para el sistema.

Capturas

Ejecución de consultas clave





Estadísticas 1

Name	Value

```
--
-- Un trigger por evento: la transition table no admite
-- declarar varios eventos en un mismo CREATE TRIGGER
CREATE TRIGGER trg_total_ins
AFTER INSERT ON detalle_pedido
REFERENCING NEW TABLE AS afectados
FOR EACH STATEMENT EXECUTE FUNCTION fn_recalcular_total();
CREATE TRIGGER trg_total_upd
AFTER UPDATE ON detalle_pedido
REFERENCING NEW TABLE AS afectados
FOR EACH STATEMENT EXECUTE FUNCTION fn_recalcular_total();
---------                    Procedimiento                    ---------
CREATE OR REPLACE PROCEDURE sp_crear_pedido(
    p_usuario_id BIGINT,
    p_forma_pago forma_pago,
    p_items      JSONB   -- [{"producto_id":1,"cantidad":2}, ...]
) AS $$
DECLARE
    v_pedido_id BIGINT;
    v_item      JSONB;
    v_producto_id BIGINT;
    v_cantidad    INTEGER;
    v_stock       INTEGER;
    v_disponible  BOOLEAN;
BEGIN
    -- El usuario debe existir y no estar eliminado
    IF NOT EXISTS (SELECT 1 FROM usuario
                   WHERE id = p_usuario_id AND eliminado = FALSE) THEN
        RAISE EXCEPTION 'Usuario % inexistente o eliminado', p_usuario_id;
    END IF;
    INSERT INTO pedido(usuario_id, forma_pago)
    VALUES (p_usuario_id, p_forma_pago)
```

GMT-03:00 | es | Editable | Inserción inteligente | 136 : 21 [4826] | Sé

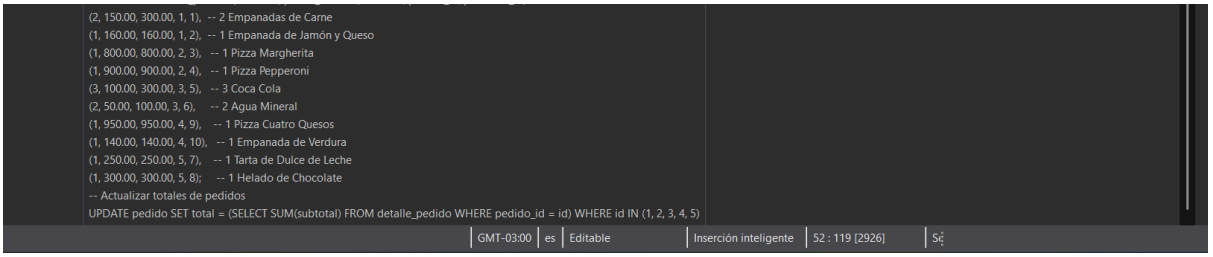
```
    RETURNING id INTO v_pedido_id;
    FOR v_item IN SELECT * FROM jsonb_array_elements(p_items) LOOP
        v_producto_id := (v_item->>'producto_id')::BIGINT;
        v_cantidad    := (v_item->>'cantidad')::INTEGER;
        -- Bloquea la fila del producto para evitar sobreventa concurrente
        SELECT stock, disponible INTO v_stock, v_disponible
        FROM producto WHERE id = v_producto_id AND eliminado = FALSE
        FOR UPDATE;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'Producto % inexistente o eliminado', v_producto_id;
        END IF;
        IF NOT v_disponible THEN
            RAISE EXCEPTION 'Producto % no disponible', v_producto_id;
        END IF;
        IF v_stock < v_cantidad THEN
            RAISE EXCEPTION 'Stock insuficiente (producto %): hay %, se piden %',
                        v_producto_id, v_stock, v_cantidad;
        END IF;
        INSERT INTO detalle_pedido(pedido_id, producto_id, cantidad)
        VALUES (v_pedido_id, v_producto_id, v_cantidad);
        -- Descuenta stock dentro de la misma transacción
        UPDATE producto SET stock = stock - v_cantidad WHERE id = v_producto_id;
    END LOOP;
    -- Si alguna inserción falla, toda la transacción se revierte (rollback).
END;
$$ LANGUAGE plpgsql
```

GMT-03:00 | es | Editable | Inserción inteligente | 136 : 21 [4826] | Sé

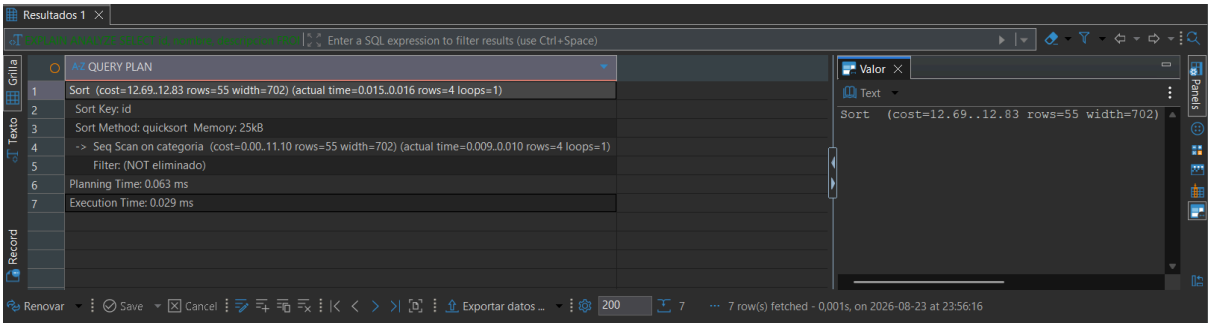
Estadísticas 1

Name	Value
Updated Rows	39
Execute time	0,165s
Start time	Sun Aug 23 23:48:54 GMT-03:00 2026
Finish time	Sun Aug 23 23:48:54 GMT-03:00 2026
Query	-- DATOS DE EJEMPLO --

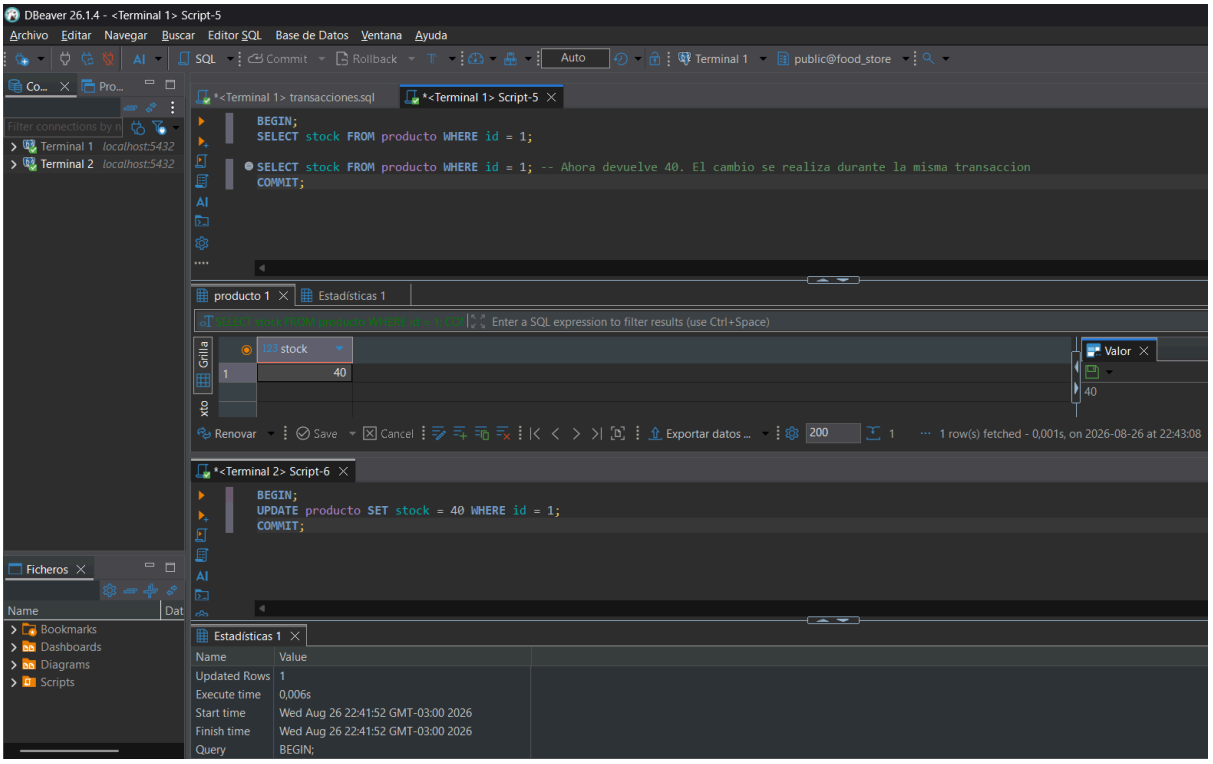
```
-- Insertar categorías
INSERT INTO categoria (nombre, descripcion) VALUES
('Empanadas', 'Variedad de empanadas'),
('Pizzas', 'Deliciosas pizzas de diferentes sabores'),
('Bebidas', 'Bebidas frías y calientes'),
('Postres', 'Dulces y postres para todos los gustos');
-- Insertar productos
INSERT INTO producto (nombre, descripcion, precio, stock, imagen, disponible, categoria_id) VALUES
('Empanada de Carne', 'Empanada rellena de carne', 150.00, 50, NULL, TRUE, 1),
('Empanada de Jamón y Queso', 'Empanada con jamón y queso', 160.00, 30, NULL, TRUE, 1),
('Pizza Margherita', 'Pizza clásica con tomate y mozzarella', 800.00, 20, NULL, TRUE, 2),
('Pizza Pepperoni', 'Pizza con pepperoni y queso', 900.00, 15, NULL, TRUE, 2),
('Coca Cola', 'Bebida gaseosa', 100.00, 100, NULL, TRUE, 3),
('Agua Mineral', 'Agua mineral sin gas', 50.00, 200, NULL, TRUE, 3),
('Tarta de Dulce de Leche', 'Deliciosa tarta de dulce de leche', 250.00, 25, NULL, TRUE, 4),
('Helado de Chocolate', 'Helado cremoso de chocolate', 300.00, 40, NULL, TRUE, 4),
('Pizza Cuatro Quesos', 'Pizza con cuatro tipos de queso', 950.00, 10, NULL, TRUE, 2),
('Empanada de Verdura', 'Empanada rellena de verduras', 140.00, 20, NULL, TRUE, 1);
-- Insertar usuarios
INSERT INTO usuario (nombre, apellido, mail, celular, contrasena, rol) VALUES
('Juan', 'Pérez', 'juan@x.com', '2611234567', 'hash1', 'USUARIO'),
('Ana', 'Garis', 'ana@x.com', '2617654321', 'hash2', 'USUARIO'),
('Luis', 'Martínez', 'luis@x.com', '2619876543', 'hash3', 'ADMIN'),
('María', 'González', 'maria@x.com', '2614567890', 'hash4', 'USUARIO'),
('Pedro', 'López', 'pedro@x.com', '2611234568', 'hash5', 'USUARIO');
-- Insertar pedidos
INSERT INTO pedido (fecha, estado, total, forma_pago, usuario_id) VALUES
```



EXPLAIN ANALYZE



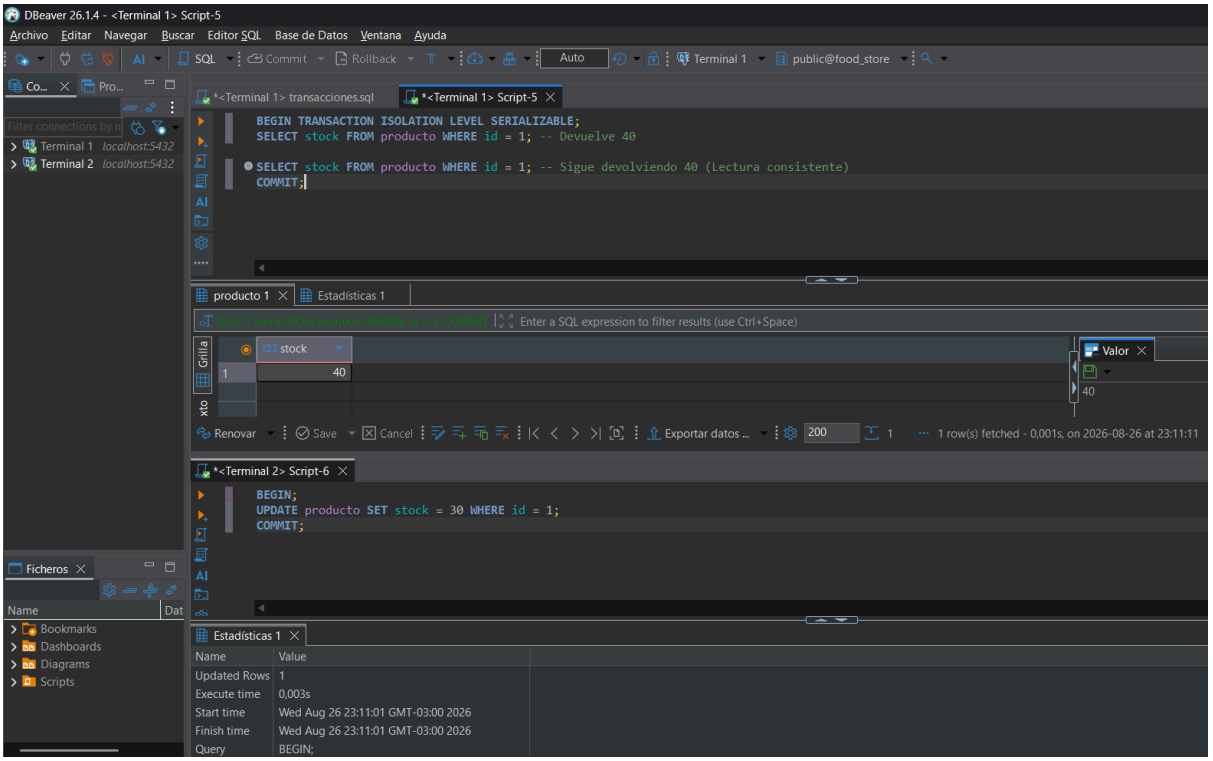
Escenarios de concurrencia



Fenómeno demostrado: Lectura No Repetible

En la captura se demuestra el comportamiento del nivel de aislamiento por defecto en PostgreSQL (Read Commited):

1. **Paso 1 (Arriba):** La **Terminal 1** inicia una transacción (Begin) y realiza una primera consulta del stock del producto.
2. **Paso 2 (Abajo):** En paralelo, la **Terminal 2** actualiza ese mismo stock a `40` y confirma los cambios guardándolos definitivamente (`Commit`).
3. **Paso 3 (Arriba):** Cuando la **Terminal 1** vuelve a consultar el mismo registro (sin haber cerrado aún su propia transacción), el valor ya no es el original, sino que cambió a `40`.



Fenómeno demostrado: Prevención de Lectura No Repetible

En la captura se comprueba el comportamiento del nivel de aislamiento más estricto de PostgreSQL (`Serializable`):

1. **Paso 1 (Arriba):** La **Terminal 1** inicia una transacción declarando explícitamente el nivel `Serializable` realiza una primera consulta del stock (devuelve `40`).
2. **Paso 2 (Abajo):** En paralelo, la **Terminal 2** actualiza ese mismo stock a `30` y confirma los cambios guardándolos definitivamente (`Commit`).
3. **Paso 3 (Arriba):** Cuando la **Terminal 1** vuelve a consultar el mismo registro, a diferencia del escenario anterior, **el valor no cambia** y sigue devolviendo `40`.

Dificultades y soluciones

Durante el desarrollo del proyecto, el principal conflicto fue en la gestión y distribución equitativa de las tareas. La solución fue mediante la delegación clara de responsabilidades, donde cada integrante asumió una parte del trabajo, optimizando así el desarrollo de este. Por otro lado, en lo que respecta al desarrollo del código y la implementación de la base de datos, no se presentaron inconvenientes técnicos de relevancia., el uso efectivo de herramientas de control de versiones y la aplicación de buenas prácticas de desarrollo, lo cual permitió un desarrollo más sencillo.

Bibliografía/Webgrafía

- Databricks. (s.f). ¿Qué es un modelo de datos relacional? https://www.databricks.com/blog/what-is-a-relational-data-model
- Cortez, A. (s. f.). Manual de conceptos de base de datos: Teoría y práctica del modelo relacional con PostgreSQL a partir del Trabajo Práctico Integrador «Food Store» [Material de clase]. Tecnicatura Universitaria en Programación, Universidad Tecnológica Nacional.

Links al repo

- https://github.com/Gonza-12-Sanchez/Tp_Base_De_Datos_FoodStore/blob/main/objects.sql